SMARTTASK: CONTEXT-AWARE MOBILE TO-DO LIST FOR INTELLIGENT TASK MANAGEMENT

De Silva S.J.W.

25-26J-531

BSc (Hons) in Information Technology Specialising in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

SMARTTASK: CONTEXT-AWARE MOBILE TO-DO LIST FOR INTELLIGENT TASK MANAGEMENT

De Silva S.J.W.

25-26J-531

Dissertation submitted in partial fulfillment of the requirements for the BSc (Hons) in Information Technology Specialising in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

We declare that this is our own work, and this dissertation does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other University or institute of higher learning, and to the best of our knowledge and belief, it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

Also, we hereby grant to the Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute our dissertation, in whole or in part in print, electronic or other mediums. We retain the right to use this content in whole or part in future works (such as articles or books).

Student ID	Name	Date	Signature
IT21228612	De Silva S.J.W.	2026/05/08	

The candidate is conducting research for their undergraduate dissertation under our supervision.

	Title	Name	Date	Signature
Supervisor	Ms	Dilani Lunugalage	06/05/2026	
Co-Supervisor	-	-	-	-
External Supervisor	-	-	-	-

Scanned with CamScanner

ABSTRACT

Task management applications are widely used by students and working professionals to organize daily activities. However, many existing applications still depend on static task lists, fixed reminders, and manual prioritization. This can increase cognitive load, cause poorly timed notifications, and result in missed tasks. To address these issues, the SmartTask application was developed as a context aware mobile task management system.

SmartTask uses contextual information such as location, time, user activity, device state, and connectivity to identify task relevance in real time. Based on this information, the system matches tasks with the user's current situation, dynamically prioritizes active tasks, and delivers notifications only when they are suitable. This helps reduce unnecessary interruptions and allows users to focus on the most relevant tasks at the right time.

The system was implemented as a native Android application using Java and Android Studio. It includes task input, context acquisition, context matching, task prioritization, adaptive notification delivery, and notification action handling. Users can create, edit, delete, complete, snooze, and postpone tasks. The application also supports local data storage, offline functionality, and privacy focused operation. Advanced behavioral learning and the full Data Logging and Insights module were not fully implemented and remain as future improvements.

The system was evaluated with 8 participants over 12 to 15 days. Users created 211 tasks and completed 185 tasks, giving an 87.7 percent task completion rate. The ignored or missed notification rate was 16.5 percent, and the overall Likert mean score was 3.73 out of 5. Overall, SmartTask proves the feasibility of combining context awareness, dynamic prioritization, and adaptive notifications in a practical mobile task management MVP.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my Research Project Supervisor, Ms. Dilani Lunugalage, for her continuous guidance, support, and encouragement throughout the development of this project. Her valuable insights and direction have played a significant role in the successful completion of this research.

I also extend my heartfelt appreciation to my former supervisor, Mr. Suresh Niroshan Fernando, for his guidance and support during the initial stages of this project. His advice and mentorship were instrumental in shaping the foundation and direction of this research.

I would further like to acknowledge the examiner panel members, Mr. Vishan Danura Jayasinghearachchi, Mr. Didula Chamara, and Mr. Eishan Weerasinghe, for their valuable feedback, constructive criticism, and continuous evaluation throughout all project milestones. Their insights greatly contributed to improving the quality and direction of this work.

I also wish to acknowledge all staff members of the Comprehensive Design and Analysis Project (CDAP) team for their support and recommendations, which were helpful in refining this project.

Finally, I am deeply thankful to my parents, friends, and colleagues for their constant encouragement and motivation throughout my academic journey.

TABLE OF CONTENTS

Declaration i

Abstract ii

Acknowledgement iii

List of figures vii

List of tables vii

List of abbreviations vii

List of Appendices viii

1 Introduction 1

2 Background & Literature Survey 3

3 Research Gap 9

3.1 Limitations of Commercial Tools 9

3.2 Limitations of Research Prototypes 9

3.3 Unaddressed Gaps in Current Literature 9

3.4 Advancement Goals and Creative Contribution of SmartTask 10

4 Research Problem 11

4.1 Specific Research Questions 11

4.2 Problem Statement 12

5 Objectives 13

5.1 Main Objective 13

5.2 Specific Objectives 13

6 Methodology 14

6.1 Methodology Overview 14

6.2 Project Requirements 14

6.2.1 Functional Requirements 14
6.2.2 Non-Functional Requirements 15
6.2.3 User Requirements 15
6.2.4 System Requirements 16
6.3 System Architecture Design 16
6.3.1 Overview of Architecture 16
6.3.2 System Components and Interactions 19
6.3.3 Data Flow of the System 20
6.4 Implementation Details 23
6.4.1 Development Environment and Technologies 23
6.4.2 Database Design 26
6.4.3 Core Algorithms and Logic 26
6.4.4 User Interface Design 28
6.5 System Workflow 29
6.6 Testing & Evaluation 30
6.6.1 Testing Approach 30
6.6.2 Evaluation Metrics 31
6.6.3 Testing Scenarios 31
6.6.4 Sample Test Cases 32
6.7 Social, Security and Ethical Considerations 32
6.7.1 Social Considerations 33
6.7.2 Security Considerations 33
6.7.3 Ethical Considerations 34
6.7.4 Limitations Related to These Considerations 34
6.8 Limitations of Implementation 35
7 Results & Discussion 36

7.1 Results 36
7.1.1 System outputs 37
7.1.2 Performance results 38
7.2 Research Findings 41
7.2.1 Key observations 41
7.3 Discussion 42
7.3.1 Interpretation of results 42
7.3.2 Strengths and limitations 43
7.4 Summary of Each Student's Contribution 45
8 Budget and Budget Justification 46
9 Ability of Commercialization 47
10 Conclusion 48
10.1 Summary of the Project 48
10.2 Achievements 48
10.3 Limitations 49
10.4 Future Work 51
10.5 Final Evaluation of the Project 51
References 53
Appendix 56

LIST OF FIGURES

Figure 1	High Level System Architecture of SmartTask	18
Figure 2	Components Flow in Real World	22
Figure 3	Category Level Results of User Evaluation Questionnaire	40

LIST OF TABLES

Table 1	Comparison of Ten Popular Commercial Apps and Ten Research Systems Along Key Dimensions	05
Table 2	Technologies and Tools Used in SmartTask	24
Table 3	Sample Test Cases Used in SmartTask	32
Table 4	User Evaluation Questionnaire Results Summary	36
Table 5	Budget Details of the Project	46

LIST OF ABBREVIATIONS

API	Application Programming Interface
CDAP	Comprehensive Design and Analysis Project
DND	Do Not Disturb
FAB	Floating Action Button
GPS	Global Positioning System
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
JDK	Java Development Kit
JSON	JavaScript Object Notation
MVP	Minimum Viable Product
QR Code	Quick Response Code
SLIIT	Sri Lanka Institute of Information Technology
SQL	Structured Query Language

SQLite	Structured Query Language Lite
UI	User Interface
UX	User Experience
ML	Machine Learning

LIST OF APPENDICES

Appendix A	Database Design	56
Appendix B	Codebase GitHub Repository Link	58
Appendix C	App UI Screenshots	59
Appendix D	User Evaluation Participation Consent Form	62
Appendix E	User Evaluation Questionnaire QR Code	64
Appendix F	User Evaluation Summary Excel Workbook Link	65
Appendix G	APK File and Installation Instructions Links	65

1 INTRODUCTION

Individuals managing tasks in fast paced digital environments experience cognitive overload due to constant multitasking and digital interruptions. Research has shown that frequent interruptions and task switching reduce performance and increase stress. For example, interrupted tasks are often completed with higher frustration and effort [1], while users frequently move between tasks in short periods, creating continuous cognitive pressure [2]. In such situations, traditional to do list applications become less effective because they present all tasks equally. This forces users to manually decide which tasks are important, increasing mental effort and reducing engagement [3].

Another common issue is notification fatigue. Users often ignore reminders because they are delivered at inappropriate times or are not relevant to their current situation. Most existing applications rely on fixed reminders that do not adapt to user context, which reduces their effectiveness over time [3].

Context aware computing has been introduced to address these problems by using real time information about the user. Context includes factors such as location, time, user activity, and device state [4]. Previous studies have shown that using context can improve usability and effectiveness in mobile systems [5][6]. Modern smartphones are capable of collecting this information through sensors and system services, making it possible for applications to respond more intelligently to user situations.

However, many existing task management tools do not fully use these capabilities. Some systems use location based reminders [7] or early context aware approaches such as Castaway [8], but they are limited to a single type of context. Other approaches such as Taskminder [9], context based filtering systems [10], and rule based assistants like DoTT [11] do not provide automatic adaptation in real world use. As a result, current solutions do not offer a complete approach for practical task management.

To address these limitations, the SmartTask application was developed as a context aware mobile task management system. It integrates multiple contextual factors such as location, time, user activity, and device state to dynamically prioritize tasks and deliver notifications at appropriate moments. The implementation focused on

validating the core concept and developing a minimum viable product that demonstrates real time context based task filtering, prioritization, and adaptive notification delivery. This behavior is supported by a modular system design that captures context, evaluates task relevance, and adjusts task ordering accordingly.

Although earlier design stages considered advanced behavioral learning and insight generation, these features were not fully implemented in the final system. They remain as potential future extensions that can enhance personalization and long term user adaptation. Previous research has shown that personalized task prioritization can reduce scheduling effort [12], and this concept can be further explored in future improvements of the system.

Overall, SmartTask demonstrates that combining context awareness with dynamic prioritization and adaptive notifications can reduce cognitive load, minimize unnecessary notifications, and improve task completion compared to traditional task management systems.

2 BACKGROUND & LITERATURE SURVEY

Modern mobile to do list applications mainly rely on static task lists and simple time based reminders. Popular tools such as Todoist allow users to create tasks easily and set due dates or location reminders, but they do not adjust task schedules automatically or reorder tasks based on changing situations [13][14]. Similarly, Any.do provides multiple reminder types such as task alerts and daily summaries, but these remain fixed and do not adapt to user behavior [15]. Microsoft To Do introduced features like My Day suggestions based on past activity, yet most task prioritization and notifications still depend on manual input [16]. Other applications such as Google Tasks and Google Keep also rely on fixed time reminders and do not provide dynamic adaptation [17][18].

Several other widely used tools focus more on organization rather than intelligent task handling. TickTick supports time and location reminders but still treats task priority as a manual setting [19]. Trello uses a board based system to manage tasks visually, but it does not automatically prioritize tasks based on urgency or context [20]. Overall, most commercial applications lack context awareness and use static prioritization with fixed notifications.

In contrast, academic research has explored more advanced approaches using context aware systems. Early systems such as Castaway provided reminders based on time and location, allowing tasks to be triggered at relevant moments [8]. ContextGear also used location data to create reminders automatically when users entered specific areas. In collaborative environments, systems like Overseer used context information to assign tasks dynamically depending on user roles and conditions [21]. These approaches showed that context awareness can improve task management, but they were often limited to specific scenarios.

Another important area of research focused on intelligent notification systems. Studies showed that using contextual information such as user activity and task phase can help determine the best time to send notifications, reducing distraction [22]. The OASIS framework introduced the idea of delaying notifications until suitable moments with lower interruption cost, which improved user performance and reduced disruption [23]. This highlighted the importance of timing when delivering reminders.

Some systems also used sensors to improve adaptability. For example, a context aware reminder system used motion sensors to detect user activities and trigger reminders accordingly [24]. The AWARE framework collected data from multiple sources such

as location, activity, and connectivity, and used this information to generate meaningful context for applications [25]. In addition, research on task prediction showed that contextual patterns can be used to estimate future tasks and improve scheduling [26].

Other studies highlighted both advantages and limitations of current approaches. The MageMemo application improved user engagement using gamification, but it did not use context awareness [3]. Another study examined how different user contexts affect interruptions and suggested strategies for better notification management [27]. Overall, the literature showed that context aware systems and adaptive notifications can improve user experience, but no single system combined all these features effectively.

The comparison of commercial applications and research systems is presented in Table 1, which highlights differences in context awareness, task prioritization, notification behavior, and learning capabilities across existing solutions.

Table 1 : comparison of ten popular commercial apps and ten research systems along key dimensions

Index	Name	Category	Context Awareness	Task Prioritization	Notification Adaptivity	Learning from Behavior	Notes
1	**Todoist**	Commercial	Partial (location)	Static	Fixed (time/location)	Yes (location suggestions)	Widely used; natural-language input[13], but only time/place triggers[14].
2	**Any.do**	Commercial	No	Static	Fixed	No	Offers standard reminders and daily summary alerts[15]; no context adaptivity.
3	**Microsoft To Do**	Commercial	No	Dynamic (My Day)	Fixed	Yes	Introduces "My Day" intelligent suggestions[16]; otherwise user-set priorities and alerts.
4	**Google Tasks**	Commercial	No	Static	Fixed	No	Bare-bones task list; only time/date reminders[17].
5	**TickTick**	Commercial	Yes (time/loc)	Static	Fixed	No	Multiple reminders (e.g. "1 day before"), supports time and *location* reminders[19].
6	**Trello**	Commercial	No	Static (board order)	Fixed	No	Kanban workflow; no smart scheduling, just due-date alerts[20].
7	**Remember The Milk**	Commercial	Yes (location)	Static	Fixed	No	Oldest list app; supports location reminders but no adaptive features.
8	**Notion**	Commercial	No	Static	Fixed	No	All-in-one workspace with checklists; lacks built-in adaptivity.
9	**Asana**	Commercial	No	Static	Fixed	No	Team task management; priorities set manually, notifications fixed.

10	**Google Keep**	Commercial	Partial (deprecated)	Static	Fixed	No	Note-taking app with reminders. Location reminders are being removed[18].
11	**Castaway**	Research	Yes (location/time)	Static	Adaptive (delay to context)	No	Prototype WIP[8]. Captures tasks quickly, reminds at right time/place.
12	**ContextGear**	Research	Yes (location)	Static	Fixed	No	Location-aware reminder prototype (Spiteri 2009).
13	**Overseer**	Research	Yes	Dynamic	Adaptive	Partial	Multi-agent system for disaster tasks[21]; uses context to reassign roles dynamically.
14	**Do Not Disturb (DND)**	Research	Yes (sensor-based)	N/A (interruptions)	Adaptive	Yes	Schedules mobile notifications at opportune breakpoints[22]; lowers interruption cost.
15	**MageMemo**	Research	No	Static	Fixed	Yes	Gamified to-do app (2024); improves engagement but is not context-aware.
16	**Context-Aware Reminder**	Research	Yes (activity)	Static	Adaptive	Yes	Android prototype uses accelerometer to detect activity for reminders[24].
17	**OASIS (Iqbal, 2008)**	Research	Yes (cognitive)	Static	Adaptive (breakpoints)	Yes	Desktop system detecting task breakpoints[23] to defer notifications; empirically reduces disruption.
18	**Task Prediction**	Research	Yes (usage context)	Dynamic	Adaptive	Yes	Tian *et al.* (2022): learns common multi-app tasks and predicts next tasks from context[26].

19	**AWARE**	Research	Yes	N/A (context platform)	N/A	Yes	Smartphone context-logging framework; plugins use data mining/ML to **generate context**[25].
20	**Kim *et al.* (2020)**	Research	Yes	N/A	Adaptive	Yes	Context-aware distraction study: derives if-then rules for proactive interruption management[27].

Based on this analysis, it was observed that commercial applications lack context awareness and dynamic prioritization, while research systems often focus on limited features. The SmartTask application was developed to address these gaps by combining multiple contextual inputs such as location, activity, and time with dynamic task prioritization and adaptive notifications. By integrating ideas from previous studies such as adaptive notification timing [23], task prediction [26], and context aware reminders [8], SmartTask provided a more complete solution for real world task management.

3 RESEARCH GAP

Despite the availability of many mobile task management applications and research studies, several important limitations were identified. These gaps were mainly related to context awareness, task prioritization, notification timing, and practical usability in real world situations.

3.1 Limitations of Commercial Tools

Commercial task management tools such as Todoist, Any.do, Microsoft To Do, TickTick, and Google Tasks are widely used, but they mostly depend on static task lists and fixed reminders [13][14][15][16][17]. Although some applications provide features like location reminders or basic suggestions, tasks still need to be manually prioritized by the user [19]. This creates additional mental effort for users who already deal with frequent interruptions and multitasking [2]. Studies have also shown that such manual management increases stress and reduces productivity [1][3]. As a result, these tools do not effectively reduce cognitive overload.

3.2 Limitations of Research Prototypes

Research prototypes have explored more advanced ideas using context aware systems, but they also have limitations. Systems like Castaway [8] and ContextGear focused on location and time based reminders, but they used only a single type of context. Other systems such as Muñoz et al. [10] and DoTT [11] introduced rule based or multi factor filtering, but they depended on user input and did not adapt automatically. Notification management systems such as OASIS [23] and Do Not Disturb [22] showed that timing notifications correctly can reduce interruptions, but these were mostly tested in controlled environments and not implemented in everyday mobile applications. Similarly, activity based reminder systems [24] and task prediction models [26] showed useful concepts but were not fully integrated into complete task management systems.

3.3 Unaddressed Gaps in Current Literature

From this analysis, several key gaps were identified. First, most systems relied on a single context type such as location [8][14] or activity [24], while real world task management requires multiple context factors to be considered together [5]. Second, task prioritization remained mostly static, with very limited automatic reordering based on real time conditions or user behavior [13][15][20]. Third, notification systems did not adapt well to user context, leading to notification fatigue due to poorly timed alerts [3][28]. Finally, although research has shown the benefits of behavior

based learning [12][26], it has not been widely implemented in practical mobile applications.

3.4 Advancement Goals and Creative Contribution of SmartTask

SmartTask addressed the identified research gaps by providing the following features.

- **Broad Context Integration:** SmartTask captured multiple contextual signals such as location, time, user activity, calendar events, and device state, moving beyond single context approaches used in earlier systems [5][8][24].
- **Dynamic Task Prioritization:** Tasks were automatically reordered based on real time context and predefined logic. This allowed more relevant tasks to be shown first, helping to reduce user effort and improve task management efficiency. Although previous research highlighted the benefits of behavior-based prioritization [12], the implemented system mainly focused on context driven prioritization.
- **Adaptive Notification Timing:** Notifications were delivered at appropriate moments based on user context. This reduced unnecessary interruptions and helped minimize notification fatigue. The design was influenced by research on adaptive notification timing such as OASIS [23] and other context aware interruption studies [27][22].
- **Behavioral Learning:** Basic user interaction handling such as task completion and notification responses were considered in the system design. However, full behavioral learning and continuous adaptation were not fully implemented in the current version. These features remain as potential improvements for future development.

By combining context awareness, dynamic task prioritization, and adaptive notifications, SmartTask provided a more practical solution compared to static commercial applications [13][17][19] and limited research prototypes [8][24][23]. The implemented system demonstrated how these features can work together in a mobile application to reduce cognitive overload and improve task relevance in real world situations [1][2][23][26].

4 RESEARCH PROBLEM

4.1 Specific Research Questions

1. How can mobile task management systems effectively integrate multiple contextual signals to enhance task prioritization?

Existing applications and research systems mostly relied on a single context factor such as location [8][14] or activity [24], along with static task ordering [13][15][20]. This did not reflect real world situations where factors such as time, location, user activity, and device state influence task relevance [5]. This study examined how these contextual signals could be combined to support dynamic task filtering and prioritization.

2. How can notifications be adaptively scheduled to minimize distraction and notification fatigue?

Most applications used fixed reminders that did not consider user context, leading to ignored or disruptive notifications [3][28][15][17]. Prior studies such as OASIS [23] and Do Not Disturb [22] showed that context based notification timing improves effectiveness. This research explored how notifications could be delivered at suitable moments based on user context.

3. How can user behavior be leveraged to personalize task management and improve adherence?

Existing applications offered limited personalization based on simple patterns such as frequent locations or daily suggestions [13][16]. Research indicated that behavior based models can improve task prioritization [12][26]. In this study, basic user interactions were considered, while advanced behavioral learning was identified as future work.

4. How can a comprehensive, context-aware mobile task management system be designed for practical use?

Previous systems demonstrated features such as location reminders [8], activity based triggers [24], or predictive models [26], but they were not combined into a single

mobile system. This study focused on designing and implementing a system that integrates context awareness, prioritization, and adaptive notifications [1][2][23][26].

4.2 Problem Statement

Individuals managing tasks in fast paced digital environments experienced cognitive overload due to multitasking, interruptions, and limitations of traditional task management systems [1][2]. Commercial applications such as Todoist, TickTick, and Microsoft To Do provided reminders and manual prioritization [13][16][19], but they did not adapt to changing user contexts. Research systems showed the value of context awareness using location and activity [8][24][23], but they were limited or not suitable for real world mobile use [22][26].

As a result, users depended on manual task management and received poorly timed notifications, increasing cognitive effort and reducing productivity [3][28]. The main issue was the lack of a unified mobile system that combines multiple contextual signals, dynamic prioritization, and adaptive notifications [5][12].

This study addressed the problem by implementing the SmartTask mobile application, focusing on core features through a working system. Advanced behavioral learning [12][26] was not fully implemented and remains future work.

Overall, the project aimed to reduce cognitive load, minimize interruptions, and improve task relevance in real world scenarios [1][2][8][23][26].

5 OBJECTIVES

5.1 Main Objective

The primary objective of this research was to design and implement a context aware mobile task management system, SmartTask, that dynamically prioritizes tasks and adapts notifications based on user context, with the aim of reducing cognitive overload and notification fatigue in everyday mobile task management**.**

5.2 Specific Objectives

1. **Contextual Signal Integration:** The objective was to identify and integrate multiple contextual inputs such as location, time, user activity, calendar events, and device status into the task management system to support context based decision making.
2. **Dynamic Task Prioritization:** The objective was to develop mechanisms that automatically reorder tasks in real time based on user context and task properties.
3. **Adaptive Notification Scheduling:** The objective was to implement a notification system that delivers alerts at appropriate moments based on user context, helping to reduce distractions and notification fatigue.
4. **Behavioral Learning:** The objective was to consider user interactions such as completed tasks and notification responses in the system design. However, continuous behavioral learning and advanced adaptation were identified as future improvements rather than fully implemented features.
5. **Evaluation of System Effectiveness:** The objective was to evaluate the effectiveness of SmartTask by observing improvements in task relevance, reduction in unnecessary notifications, and overall user experience.
6. **Broad System Design:** The objective was to design a practical mobile application that integrates context awareness, task prioritization, and adaptive notifications into a single system suitable for everyday use.

6 METHODOLOGY

6.1 Methodology Overview

The project followed a practical development approach focused on building a context aware mobile task management application. The system was implemented as a native Android application using a modular design, where separate components handled task input, context acquisition, processing, and notification delivery.

A prototype-based approach was used to develop a minimum viable product. The implementation focused on key features such as context acquisition, task prioritization, and adaptive notifications. This allowed the core concept to be tested in real world scenarios. Advanced features such as detailed behavioral learning and data insights were considered during the design stage but were not fully implemented and remain as future improvements.

A context aware design was selected because task relevance depends on changing user conditions such as location, time, activity, and device state. The system captured this information and used it to filter tasks, assign priorities, and deliver notifications at appropriate moments. The overall system followed a pipeline where context data was collected, matched with tasks, and used to support prioritization and notification decisions.

Overall, this methodology enabled the development of a working system that demonstrates how context awareness can improve task management and reduce unnecessary interruptions.

6.2 Project Requirements

6.2.1 Functional Requirements

- Task Management: Users were able to create, edit, and delete tasks with details such as title, deadline, priority, and location.
- Context Acquisition: The system collected real time contextual information including location, time, user activity, and device state using mobile sensors and system services.

- Context Matching: Captured context data was matched with task information to identify relevant tasks based on the current situation.
- Task Prioritization: Tasks were automatically reordered based on urgency and context relevance, allowing important tasks to be shown first.
- Adaptive Notification System: Notifications were triggered at appropriate moments based on user context, while unnecessary alerts were suppressed to reduce interruptions.
- Additional Design Consideration: A data logging and insights component was identified during the initial design stage to support behavioral learning. However, this feature was not implemented in the current system and remains as potential future enhancement.

6.2.2 Non-Functional Requirements

- Performance: The application provided near real time responses for task updates and notifications.
- Usability: A simple and user friendly interface was implemented to ensure ease of use.
- Reliability: The system functioned consistently with offline support for core features.
- Privacy: All user data was stored locally on the device to protect user privacy.
- Efficiency: The system minimized battery and resource usage by controlling background processing.

6.2.3 User Requirements

User requirements describe what the end-user expects from the system.

- Easy creation and management of tasks
- Receiving relevant and timely notifications
- Avoiding unnecessary interruptions
- Ability to use the application without internet connectivity
- Minimal effort required to manage daily tasks

6.2.4 System Requirements

- Hardware: Android smartphone with GPS and activity recognition capabilities
- Software: Android Studio development environment with Java programming language
- Platform Services: Access to device sensors such as location, activity recognition, and system state
- Database: Local database using SQLite or Room for storing tasks and context data

6.3 System Architecture Design

6.3.1 Overview of Architecture

The SmartTask system was designed using a modular architecture that separates the system into multiple components, each responsible for a specific function. This design made the system easier to develop, maintain, and extend. Each module handled a distinct part of the task management process, allowing the system to process user input, collect context data, and generate appropriate outputs efficiently.

The overall system followed a structured processing pipeline where contextual information was continuously collected and used to support decision making. This pipeline can be summarized as: context acquisition, context matching, task prioritization, and notification delivery. Context data such as location, time, user activity, and device state was captured first. This information was then matched with task details to determine relevance. Based on this, tasks were dynamically prioritized, and notifications were triggered at suitable moments.

The architecture consisted of six main components as identified during the design phase. These included task input, context acquisition, context matching, task prioritization, adaptive notification, and a data logging component. However, the data logging and insights component was not implemented in the final system and was considered as a possible future enhancement. The implemented system focused on the core pipeline of capturing context, processing relevance, and delivering notifications in real time.

The modular structure also supported an offline first design, where all data processing and storage were handled locally on the device. This improved reliability and ensured user privacy, as no external data transmission was required. Additionally, the architecture allowed different components to interact through a clear flow of data, supporting real time updates and efficient system behavior.

The overall system architecture is illustrated in Figure 1, which shows how each component interacts within the SmartTask system.

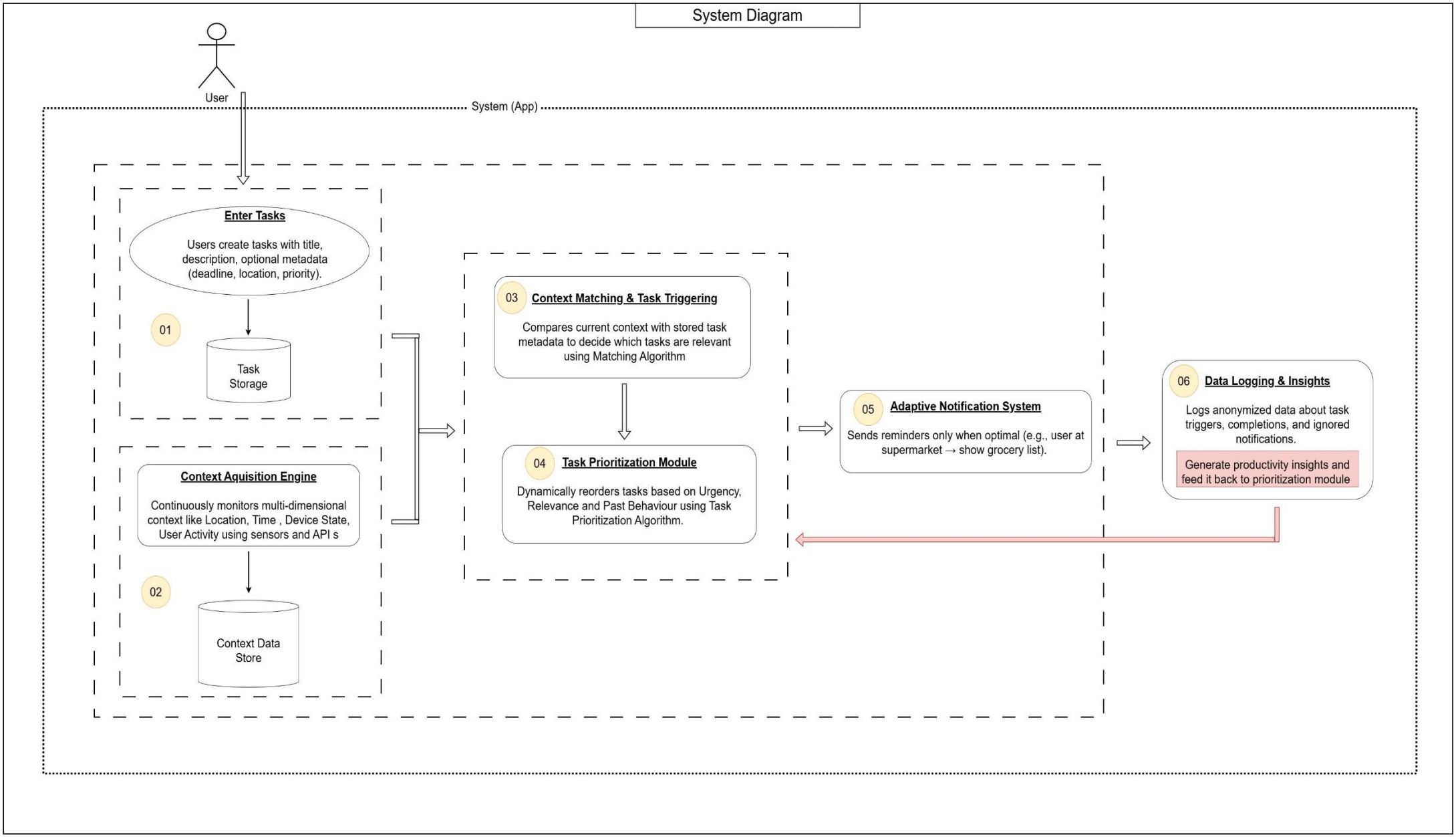


Figure 1 : High-Level System Architecture of SmartTask

6.3.2 System Components and Interactions

As depicted in Figure 1, the SmartTask system was designed using a set of modular components, each responsible for a specific function in the task management process. These components worked together to capture user input, process contextual data, and deliver relevant outputs.

1. Task Input Module

This module handled user interactions related to task management. Users were able to create, edit, and delete tasks with details such as title, deadline, priority, and location. Instead of using a basic Low, Medium, and High priority scale, the module used five priority choices: Critical now, High, Important, Routine, and Low. This provided a clearer way to capture urgency, importance, and consequence during task creation. Research on context-aware task prioritization shows that richer task signals can improve task ranking compared with simple priority labels [12]. It also reduces the ambiguity of "Medium" by separating planned valuable work from ordinary routine tasks. Since poorly prioritized mobile notifications can increase stress and workload, clearer priority input supports better task ordering and reminder decisions [1] [29]. The module also supported structured task storage and ensured that all task data was saved in the local database. In addition, features such as voice-based task input were included to improve usability.

2. Context Acquisition Engine

This component was responsible for collecting real time contextual data from the device. It gathered information such as location, time, user activity, and device state using available sensors and system services. The collected data was stored as context snapshots, which were then used by other components for processing.

3. Context Matching Module

The context matching component analyzed the relationship between the current context and stored task data. It evaluated which tasks were relevant based on factors such as proximity to location, time remaining to deadline, and user activity. This

filtering process helped identify tasks that should be considered for prioritization and notification.

4. Task Prioritization Module

This module ranked tasks based on their importance and relevance. It considered factors such as urgency, context match, and user defined priority to determine the order of tasks. The system updated the display order dynamically so that the most relevant tasks appeared at the top of the list.

5. Adaptive Notification System

This component handled the delivery of notifications to the user. Notifications were triggered only when certain conditions were met, such as sufficient task relevance and acceptable user interruption level. The system also supported actions such as marking tasks as completed, snoozing, or postponing tasks directly from notifications.

6. Data Logging and Insights Component

This component was part of the initial system design and was intended to record user interactions and generate insights for improving system behavior. However, it was not implemented in the final version of the system. It remains as a potential extension for future development.

These components were connected through a continuous processing flow where context data was collected, matched with tasks, used for prioritization, and finally applied in notification delivery. This modular structure supported efficient system operation and allowed the core functionality to be implemented as a working mobile application.

6.3.3 Data Flow of the System

The SmartTask system followed a structured data flow where information moved through multiple components to support task prioritization and notification delivery. The process began with user input and continued through context collection, processing, and output generation.

- First, tasks were created by the user through the task input module. Each task included details such as title, deadline, priority, and optional location. This

information was stored in the local database and used as the base input for further processing.

- Next, the context acquisition engine continuously collected real time data from the device. This included information such as current location, time, user activity, and device state. The collected data was stored as context snapshots, which represented the current situation of the user.
- The context matching module then compared the current context with stored task data. It identified which tasks were relevant based on factors such as proximity to task location, time remaining to deadline, and user activity. This step filtered the task list to highlight the most suitable tasks for the current situation.
- After that, the task prioritization module assigned priority scores to tasks based on urgency and context relevance. Tasks were reordered dynamically so that the most important and relevant tasks appeared at the top of the list. This ensured that users could easily focus on tasks that required immediate attention.
- Finally, the adaptive notification system evaluated whether a notification should be triggered. Notifications were sent only when certain conditions were met, such as high task relevance and acceptable interruption level. This helped reduce unnecessary alerts and improved user experience.

A real-world example of this data/component flow is illustrated in Figure 2. In this scenario, a student had multiple tasks with different deadlines and locations. As the student moved closer to a relevant location and the deadline approached, the system identified the most important task, prioritized it, and delivered a timely notification.

The step-by-step flow illustrated in Figure 2 shows how SmartTask processed data in real time to support intelligent task management and reduce cognitive effort for the user.

Scenario: A student has 3 upcoming tasks today:

1. Finish Assignment – Deadline: 2:00 PM | Location: Library | Priority: High
2. Buy Stationery – Deadline: Evening | Location: Bookshop | Priority: Medium
3. Call Family – Deadline: Night | Location: Home | Priority: Low

1 - Task Input Module

- Student enters tasks with metadata: title, deadline, location, priority.
- Example: "Finish Assignment – 2:00 PM – Library – High Priority".
- Stored in Task Database.

2 - Context Acquisition Engine

- Continuously gathers real-time signals:
  - Location → detects when student is at home or library.
  - Time → checks system clock (e.g., 1:30 PM, 30 mins before deadline).
  - Device state → battery at 20% triggers early reminder.
  - Activity → detects "walking" → student is moving toward library.
- Stored in Context Data Store.

3 - Context Matching & Task Triggering

- Matches current context with task metadata.
- Example: At 1:30 PM, location = near library, time = 30 mins to deadline.
- Filters task list → highlights "Finish Assignment" as relevant.

4 - Task Prioritization Module

- Orders tasks dynamically:
  - Finish Assignment → top priority (deadline + location match).
  - Buy Stationery → lower (not urgent now).
  - Call Family → lowest (later at night).
- Works in loop with insights to refine ranking.

5 - Adaptive Notification System

- Sends just-in-time reminder:
  - "You're near the library, time to finish your assignment before 2:00 PM."
- Suppresses irrelevant notifications (Call Family won't pop up now).

6 - Data Logging & Insights

- Logs: student opened the reminder, completed assignment at 1:55 PM.
- Stores response + context.
- Feeds back into prioritization → system learns optimal reminder timing (e.g., 30 mins before worked well).

Result:

- Student finishes assignment on time with minimal distractions.
- Notifications felt timely and useful.
- System adapts to user behavior for future days.

Figure 2 : Components Flow in Real World

6.4 Implementation Details

6.4.1 Development Environment and Technologies

The SmartTask application was developed as a native Android application using Android Studio as the main development environment. Java was selected as the programming language due to its strong support for Android development and full access to system level APIs.

The system used Android native services such as location services, activity recognition, and notification APIs to collect contextual data and deliver notifications. These technologies enabled the application to operate efficiently in real time while maintaining compatibility with mobile devices.

The technologies and tools used in the system are summarized in Table 2.

Table 2 : Technologies and Tools Used in SmartTask.

#	Task / Requirement	Available Technologies / Tools	Selected Technology & Justification
1	Mobile App Development	Java, Kotlin, Flutter, React Native	Java (Android native) – Provided full access to Android APIs and ensured stable performance for sensor-based features.
2	Context Acquisition	Google Maps API, Android Location Services, Activity Recognition API	Android native APIs + Google Maps API – Enabled accurate location tracking, activity detection, and context collection.
3	Data Storage	SQLite, Room, Firebase	SQLite / Room – Lightweight and efficient for storing tasks and context data locally without network dependency.
4	Context Matching	Java libraries, rule engines	Custom Java logic – Simple and efficient for matching tasks with real time context.
5	Task Prioritization	Custom algorithms, rule engines	Custom scoring logic – Allowed dynamic task ordering based on urgency and context relevance.
6	Notification System	Android Notification API, Firebase Cloud Messaging	Android Notification API – Supported local, context aware notifications without requiring internet access.
7	Behavioral Learning & Insights	SQLite, Firebase Analytics, ML libraries	Not fully implemented – Identified in design phase but left as future improvement.
8	Testing	Android Emulator, Physical Devices	Emulator + Real Devices – Used for validating UI, sensors, and system behavior.

9	Testing Frameworks	JUnit, Espresso, Mockito	JUnit + Espresso – Used for basic testing of logic and user interface.
10	Development Environment	Android Studio, VS Code	Android Studio – Official IDE with full support for Android development.
11	Version Control	Git, GitHub	Git + GitHub – Used for version management and code tracking.
12	UI Design	Figma, Adobe XD	Figma – Used for designing user interface layouts before implementation.
13	Hardware	Android devices, PC	Android devices + PC – Required for testing sensors and running the application.
15	Voice Command Parsing and analysing, Task description analysing for scoring logics (Those are new features suggested by examiners)	Open AI API, Google Gemini, Local ML Models	Open AI API – Easiness of integration for the MVP. Also, Local ML Models will increase resource usage of the mobile phone.

6.4.2 Database Design

The SmartTask system used a local database implemented with Room over SQLite to store application data. This ensured fast data access and offline functionality.

The main data structures included:

- Task Data: Stored task details such as title, deadline, priority, location, and completion status
- Context Snapshots: Stored contextual information such as time, location, activity, and device state
- Matching Results: Stored intermediate results used for evaluating task relevance

The detailed database schema, including all tables and attributes used in the SmartTask system, is provided in Appendix A.

6.4.3 Core Algorithms and Logic

The SmartTask system used a structured processing pipeline to manage context based task prioritization and notification delivery. The main logic was divided into context acquisition, context matching, task prioritization, and notification decision making.

1. Context Acquisition Logic

- The system captured context when the app started, during scheduled background checks, and when geofence events occurred.
- The ContextEngine created a context snapshot for each capture event.
- Context collectors gathered data such as time, permission state, geofence status, location, activity, calendar state, device state, connectivity, and environmental details.
- The collected data was stored as a ContextSnapshot in the local database.
- A ContextDeriver calculated useful values such as interruption cost, receptivity score, and context label.

2. Context Matching Logic

- After a context snapshot was created, the ContextMatchWorker started the matching process.
- The system selected active tasks that were not completed, not archived, not snoozed, and had notifications enabled.
- Tasks with location data were checked against the current user location.
- Each task was evaluated using factors such as location proximity, deadline urgency, manual priority, calendar status, user activity, and device state.
- The final relevance score was limited within a fixed range to keep scoring consistent.
- Tasks could be blocked from notification during unsuitable conditions such as meetings, driving, low battery, offline status, Do Not Disturb mode, or cooldown periods.

3. Task Prioritization Logic

- The TaskPrioritizationRunner ranked all active incomplete tasks.
- Each task received a final score based on urgency, context relevance, interruption cost, and manual priority.
- Urgency was calculated using deadline closeness and overdue status.
- Context relevance was calculated using factors such as location, time, activity, and device fit.
- Interruption cost reduced the score when the user was in a busy or unsuitable context.
- The final task order was saved using the displayOrder value in the database.
- Small score differences were ignored to reduce unnecessary task list changes.

4. Notification Decision Logic

- The notification system checked whether a task was suitable for notification.
- A notification was sent only if the task had a valid notification trigger.
- The task had to be unblocked and meet the required score level.
- The system also checked the interruption penalty before sending alerts.
- Only a limited number of notifications were allowed in one processing cycle.

- This helped prevent notification overload.

5. Notification Action Handling

- The system supported quick notification actions such as Done, Snooze, and Postpone.
- Selecting Done marked the task as completed.
- Selecting Snooze delayed the reminder for a later time.
- Selecting Postpone moved the due time forward.
- These actions updated the task state and affected future prioritization decisions.

Overall Processing Flow is summarized below.

- Context Acquisition produced a context snapshot.
- Context Matching calculated task relevance.
- Task Prioritization updated the order of active tasks.
- Notification Decision selected suitable tasks for alerts.
- User actions updated task data for later processing.

This logic allowed SmartTask to operate as a lightweight context aware system. It did not implement full behavioral learning, but user actions such as completing, snoozing, and postponing tasks influenced future task handling through updated task states. The complete source code of the SmartTask application is available through the GitHub repository provided in Appendix B.

6.4.4 User Interface Design

The user interface was designed to be simple and user friendly. Sample user interface screens of the SmartTask application are included in Appendix C for visual reference.

- Task List Screen: Displayed tasks in prioritized order and allowed quick actions such as complete or delete. Includes voice command task creation and normal task creation buttons as floating action buttons (FAB).
- Task Creation Screen: Allowed users to add and edit tasks with required details
- Notification Interaction: Allowed users to complete, snooze, or postpone tasks directly from notifications

6.5 System Workflow

The SmartTask system workflow explains how the user interacts with the application and how the system responds in a normal task management scenario. This section focuses on the step-by-step user experience.

- User Opens the Application : The workflow starts when the user opens the SmartTask application. The app displays the main task list screen, where existing tasks are shown in prioritized order. During startup, the system also prepares background services such as context collection and notification handling.
- User Creates or Updates a Task : The user can create a new task manually or through the voice task input feature. Task details such as title, description, category, priority, due date, preferred time, location, and notification settings can be added. If a location is selected, the map picker helps choose the correct place. After saving, the task is stored in the local database.
- System Captures Current Context : After task creation and during normal app usage, the system captures the current user context. This includes time, location, user activity, calendar state, device state, and connectivity. Context capture can occur during app startup, periodic background checks, or geofence related events.
- System Matches Tasks with Context : The captured context is compared with stored task details. For example, if a task is linked to a specific location and the user is near that place, the task becomes more relevant. The system also considers urgency, activity, device state, and possible conditions such as meetings, driving, low battery, or Do Not Disturb mode.
- System Prioritizes the Task List : After context matching, the system calculates task priority using urgency, context relevance, interruption cost, and manual priority. The task list is then reordered so that the most relevant and urgent tasks appear first, while maintaining stability to avoid unnecessary frequent changes.
- System Sends Suitable Notifications : The notification system evaluates whether a task should be shown to the user. A notification is sent only when

the task is relevant, not blocked, and suitable for the user's current situation. The system also limits the number of notifications to reduce unnecessary interruptions.

- User Responds to Notification : The user can interact with the notification by selecting actions such as Done, Snooze, or Postpone. Selecting Done marks the task as completed, Snooze delays the reminder, and Postpone adjusts the due time. These interactions are stored and influence later task processing.
- Workflow Continues Dynamically : As the user's context changes over time, the system continuously updates task relevance, task ordering, and notification decisions. This allows SmartTask to provide a more adaptive task management experience compared to traditional fixed reminder systems.

6.6 Testing & Evaluation

6.6.1 Testing Approach

SmartTask is tested using unit testing, UI testing, integration testing, and real device validation to ensure correct system logic, behaviour, and user interaction.

- Unit testing (JUnit) verifies core logic such as task validation, context matching, prioritization, and notification rules, ensuring functions produce correct outputs.
- UI testing (Espresso) checks user interactions including task creation, editing, deletion, completion, and navigation to ensure proper system response.
- Integration testing (Mockito) simulates interactions between modules using controlled context inputs (location, time, activity, device state), validating end-to-end system functionality.
- Testing is performed on both Android Emulator and real devices. The emulator simulates conditions, while real devices validate sensor-based features like location tracking, activity recognition, and notifications.
- User testing is conducted with selected users (students and working professionals) who use the app in daily routines over a two-week period, ensuring realistic evaluation. All participants provided informed consent

before taking part in the evaluation, and the consent form used is included in Appendix D.

- Feedback is collected through a Google questionnaire focusing on usability, prioritization, notifications, and overall satisfaction, followed by discussions for deeper insights. The questionnaire used for collecting user feedback was distributed through a QR code, which is provided in Appendix E.

6.6.2 Evaluation Metrics

SmartTask is evaluated using both performance and user experience metrics.

- **Task Completion Rate** : measures how many tasks are completed within the testing period.
  Task Completion Rate = Completed Tasks / Total Tasks × 100
  A higher rate indicates effective prioritization and timely task support.

- **Missed Reminder Rate** : evaluates notification effectiveness.
  Missed Reminder Rate = Ignored Notifications / Total Notifications × 100
  A lower rate shows that notifications are relevant and well-timed.

- **User Satisfaction** is measured through questionnaire ratings on usability, clarity, and usefulness, along with qualitative feedback from discussions.

- **System Responsiveness** measures how quickly the system updates tasks and triggers notifications after context changes.

6.6.3 Testing Scenarios

Testing scenarios simulate real-life usage to validate system behavior under different contexts.

- **Context-based scenarios** check whether tasks are correctly identified based on location, time, and activity.

- **Notification scenarios** verify that alerts are delivered at appropriate times and suppressed when necessary.

- **Prioritization scenarios** ensure tasks are dynamically reordered based on urgency and relevance.

- **User interaction scenarios** validate actions like creating, editing, and completing tasks.
- **End-to-end scenarios** test the full workflow from task creation to notification delivery.

6.6.4 Sample Test Cases

Sample Test Cases used in the system are summarized in Table 3.

Table 3 : Sample Test Cases Used in SmartTask.

Test Case ID	Test Scenario	Description	Test Steps	Expected Result
TC01	Task Creation	Verify that a user can create a task with valid input	1. Open task creation screen 2. Enter task details 3. 3. Save task	Task is stored successfully and displayed in task list
TC02	Context-Based Triggering	Verify that tasks are identified as relevant based on user context	1. Create task with location and time 2. Simulate user in that location	Task is highlighted and prioritized
TC03	Adaptive Notification	Verify that notifications are delivered at appropriate times	1. Create task with deadline 2. Simulate suitable user state	Notification is triggered at correct time
TC04	Task Prioritization	Verify that tasks are ordered based on urgency and context	1. Create multiple tasks 2. Simulate context changes	Tasks are dynamically reordered
TC05	Task Completion	Verify that completing a task updates the system correctly	1. Mark task as completed 2. Observe task list	Task is updated and marked as completed

6.7 Social, Security and Ethical Considerations

Since SmartTask uses personal task information and contextual data, social, security, and ethical aspects were considered during the design, implementation, and evaluation of the system. These considerations were important because the application handled

user related information such as task details, location, time, activity state, device state, and notification interactions.

6.7.1 Social Considerations

The SmartTask application was developed to support students, working professionals, and other daily task users who manage multiple responsibilities. The system aimed to reduce mental effort by showing relevant tasks and delivering reminders at suitable moments. This supports users who experience difficulty managing tasks manually or who often miss reminders due to busy routines.

At the same time, the system was designed to avoid unnecessary interruption. Since frequent notifications can disturb users during meetings, travel, study sessions, or work, SmartTask included notification decision rules to reduce unsuitable alerts. The application considered conditions such as meetings, driving, low battery, Do Not Disturb mode, and user activity before sending reminders. This helped reduce notification overload and supported more responsible use of mobile notifications.

6.7.2 Security Considerations

The SmartTask system followed a local storage approach. Task data and context related data were stored on the user's device using a local database instead of being shared through an external cloud service. This reduced the risk of exposing sensitive user information to third party services.

The application also depended on Android permission controls for sensitive data access. Permissions such as location, activity recognition, calendar access, notification permission, and audio recording were required only for relevant features. If a required permission was denied, the related function was limited rather than forcing access. This helped ensure that users had control over the data collected by the application.

The system also minimized external data sharing. Core task management, context matching, prioritization, and notification handling were processed locally. This was important because task details and contextual information can reveal private user routines, locations, and personal responsibilities.

6.7.3 Ethical Considerations

Ethical use of data was considered during both implementation and user evaluation. The application collected contextual data only to improve task relevance, prioritization, and notification timing. The collected data was not used for unrelated purposes or shared with external parties.

User consent was also considered during evaluation. Participants were informed about the purpose of the testing process and the type of feedback collected. The consent form used for user evaluation is included in Appendix D. The questionnaire used for collecting user feedback was shared through the QR code provided in Appendix E.

The system also considered transparency and user control. Users were able to create, edit, delete, complete, snooze, and postpone tasks. They were also able to control notification related behavior through task settings. However, user evaluation feedback showed that the prioritization logic should be explained more clearly in the interface. Therefore, explanation labels such as due soon, near location, high priority, or suitable time were identified as an important improvement.

6.7.4 Limitations Related to These Considerations

Although the system used local storage and permission-based access, some limitations remained. First, context aware features depended on device permissions and sensor availability. If location or activity permissions were denied, the system could not fully identify the user's context. Second, location accuracy and background processing limitations could affect notification timing. Third, the completed MVP did not include a full privacy dashboard or detailed data deletion interface for all stored context records. Fourth, the full Data Logging and Insights module was not implemented as a user facing analytics feature, so advanced user control over behavioural insights was not available.

Overall, SmartTask addressed the main social, security, and ethical concerns by using local data storage, permission-based access, user consent during evaluation, and notification suppression rules. However, future improvements should include clearer prioritization explanations, stronger privacy controls, and more transparent management of stored context data.

6.8 Limitations of Implementation

SmartTaAlthough the SmartTask system successfully demonstrated the core concept of context aware task management, several limitations were identified in the current implementation.

- Limited Behavioral Learning: The system captures user actions (complete, snooze, postpone) but does not implement advanced learning to adapt automatically over time.
- No Data Logging and Analytics Module: The planned data logging and insights component was not implemented, so detailed analytics such as task trends and productivity insights are not available.
- Limited Real-World Testing: Testing was mainly conducted using controlled scenarios, emulator simulations, and limited real device usage, without large-scale real-world evaluation.
- Dependency on Sensors and Permissions: The system relies on device sensors and permissions (e.g., location, activity). If unavailable or denied, some features may not function properly.
- Battery and Resource Usage: Continuous context monitoring, especially location tracking, may increase battery consumption on some devices.
- MVP Scope Limitations: The system was developed as a minimum viable product, so advanced features such as cloud integration, cross-device sync, and deeper personalization were not included.

Overall, these limitations reflect the focus on validating the core concept, while future improvements can enhance scalability, adaptability, and real-world performance.

7 RESULTS & DISCUSSION

7.1 Results

This chapter presents the results obtained from the completed SmartTask application and the user evaluation questionnaire. The evaluation focused on identifying whether the SmartTask MVP successfully supported context aware task management, adaptive reminders, and practical daily task usage. The results are mainly based on system outputs, usage records, Likert scale responses, and open-ended user feedback collected from the questionnaire.

A Likert scale is a commonly used rating method in questionnaires where participants express their level of agreement with a statement. In this evaluation, a five point Likert scale was used, ranging from Strongly Disagree to Strongly Agree. This method was suitable for the SmartTask evaluation because it helped measure user opinions about usability, notification usefulness, context awareness, task prioritization, satisfaction, and perceived productivity in a simple and structured way. Since the participant sample was small, Likert scale results were analyzed using descriptive statistics such as mean score, median score, and response counts rather than advanced statistical testing.

User evaluation results are summarized in Table 4 below. The complete dataset, including raw responses, calculated metrics, and detailed analysis, is available in the Excel workbook provided in Appendix F.

Table 4 : User evaluation questionnaire results summary

Metric	Value
Participants	8
Total active testing days	109
Average active days per participant	13.62
Active days range	12 to 15
Total tasks created	211
Average tasks created per participant	26.38

Total tasks completed	185
Average tasks completed per participant	23.12
Overall task completion rate	87.7%
Mean participant completion rate	88.6%
Completion rate range	81.6% to 96.2%
Total notifications received	194
Average notifications received per participant	24.25
Total notifications ignored or missed	32
Average ignored or missed notifications per participant	4
Overall ignored or missed notification rate	16.5%
Mean participant ignored or missed rate	16.2%
Ignored or missed rate range	9.1% to 28.1%
Overall Likert mean	3.73

The evaluation was conducted with 8 valid participants. The participant group included 4 students and 4 working professionals. All participants had previous experience using to do list applications before using SmartTask. This helped to collect feedback from users who were already familiar with task management tools and could compare SmartTask with their previous experience.

The participants used the application for 12 to 15 days, with an average active usage period of 13.63 days. During this period, users created tasks, interacted with notifications, completed tasks, and provided feedback about usability, context aware features, notification usefulness, and overall satisfaction.

7.1.1 System outputs

The completed SmartTask application produced the main outputs expected from the MVP. The system allowed users to create and manage tasks with important details such as title, description, priority, due date, preferred time, and location. The application also supported task editing, deletion, completion marking, manual ordering, and notification based actions such as done, snooze, and postpone.

The context acquisition process captured user related conditions such as time, location, activity, device state, and connectivity. These context values were used by the matching and prioritization process to identify which tasks were more relevant at a particular moment. The system then updated the task order based on urgency, contextual relevance, and interruption conditions. This helped the app show more suitable tasks instead of showing a completely static task list.

The adaptive notification output was another important result of the system. SmartTask delivered notifications when task conditions matched the current user context. For example, a location based task could become more relevant when the user was near the related place, while a task with an upcoming deadline could receive higher priority when the due time was close. The notification process also considered blocking conditions such as meetings, driving, low battery, and do not disturb status before deciding whether a notification should be shown. This follows the completed app design, where matching, prioritization, and notification delivery operate as one pipeline.

The final MVP also included basic interaction handling. When users responded to notifications by marking a task as done, snoozing it, or postponing it, the task state was updated and affected later task ordering and reminder decisions. However, the full Data Logging and Insights module described in the original proposal was not implemented as a complete user facing analytics feature. Therefore, this report treats advanced analytics and detailed productivity insight generation as an extension of the project rather than a completed feature.

7.1.2 Performance results

The user evaluation showed positive task usage and completion results. Across all 8 participants, a total of 211 tasks were created during the testing period. Out of these, 185 tasks were completed. This gives an overall task completion rate of 87.7 percent.

This result indicates that participants were able to use SmartTask successfully for daily task management. A high task completion rate is important because one of the main goals of SmartTask was to support users in completing tasks more effectively by showing relevant tasks and providing context based reminders.

Notification engagement also showed positive results. Participants received a total of 194 notifications during the evaluation period. Out of these, 32 notifications were ignored or missed. This gives an ignored or missed notification rate of 16.5 percent. This means most notifications received user attention, while a smaller portion was missed or ignored.

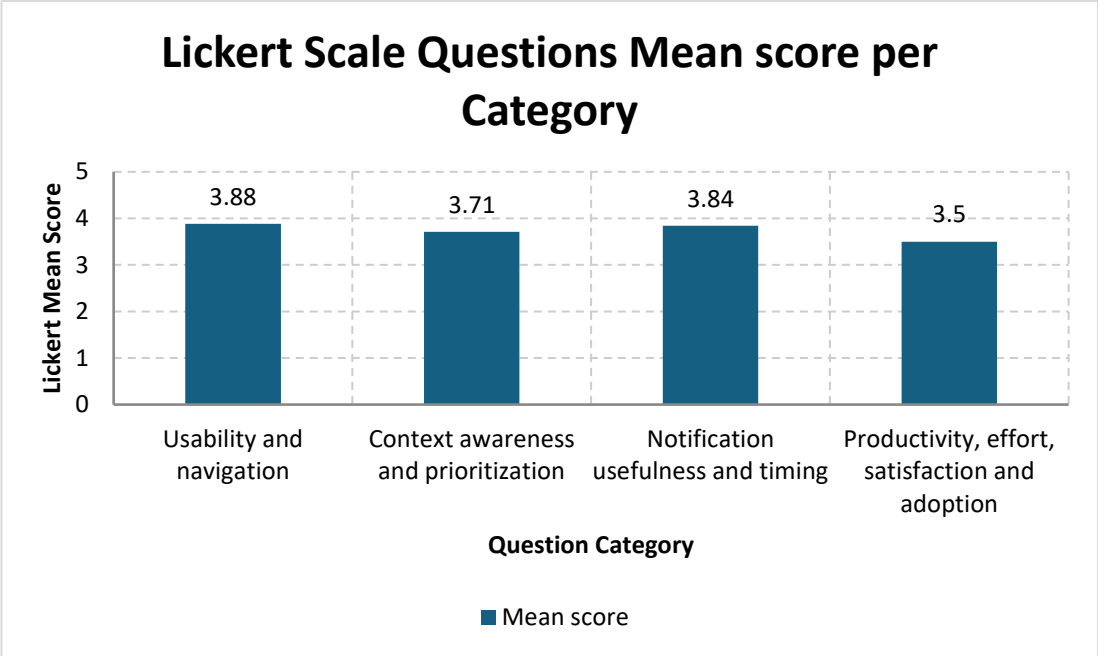
The Likert scale results also supported the usability and usefulness of the application. The questionnaire included 15 Likert scale items. The overall mean score across all Likert items was 3.73 out of 5. No participant selected a negative rating for any Likert statement. The responses were mainly Agree and Neutral, showing that the users generally accepted the application, while still identifying areas that need improvement.

The category level results were as follows:

Figure 3 summarizes the category level results of the Likert scale questions in the user evaluation questionnaire. Each category included 3 to 4 related questions, rated using the following five-point scale:

- 1 - Strongly Disagree
- 2 - Disagree
- 3 - Neutral
- 4 - Agree
- 5 - Strongly Agree

Figure 3 : Category level results of user evaluation questionnaire



Usability and navigation recorded the highest mean score of 3.88 out of 5. This shows that users generally found the application simple enough to understand and use.

Notification usefulness and timing recorded a mean score of 3.84 out of 5. This suggests that users found the reminder system useful, especially when reminders were linked to time and location.

Context awareness and task prioritization recorded a mean score of 3.71 out of 5. This supports the main concept of SmartTask, although some feedback showed that users wanted more explanation about why certain tasks were ranked higher.

Productivity, effort, satisfaction, and adoption recorded a mean score of 3.50 out of 5. This was the lowest category mean, but it was still not negative. This shows that users saw value in the application, but the short evaluation period was not enough to strongly prove long term productivity improvement.

The highest scoring individual areas were navigation smoothness and notification timing, both with a mean score of 4.13 out of 5. The lowest scoring item was perceived

productivity improvement, with a mean score of 3.38 out of 5. This indicates that the app was easier to accept as a usable and helpful tool, but stronger evidence is needed before claiming major productivity improvement.

7.2 Research Findings

The research findings show that SmartTask achieved the main purpose of the MVP. The application proved that a mobile to do list can use user context to improve task relevance and reminder timing. The results do not prove that SmartTask fully solves productivity issues, but they show that the concept is practical and useful for real users.

7.2.1 Key observations

The first key observation is that users responded positively to context aware reminders. All participants stated that the context aware features were useful in some way. Many users liked that the app reminded them about tasks when they were at a relevant place or when the task became more suitable based on time. This supports the main research idea that reminders should not only depend on fixed time settings.

The second observation is that the task completion rate was strong. The completion of 185 tasks out of 211 created tasks shows that users actively used the app and completed most of their planned work. This result supports the usefulness of SmartTask as a task management MVP.

The third observation is that notification engagement was generally positive. The ignored or missed notification rate was 16.5 percent, meaning that most notifications were noticed or acted on. This shows that adaptive reminders can improve user attention when compared with reminders that appear at random or unsuitable times.

The fourth observation is that usability was one of the strongest areas of the system. The highest category mean was recorded for usability and navigation. This is important because a context aware system can fail if the interface is too complex. In this evaluation, users were able to understand the basic app flow and use it for daily tasks.

The fifth observation is that users wanted more transparency in task prioritization. Some users were not fully sure why a certain task appeared higher than another task. This shows that the ranking system needs a clearer explanation in the interface. A

simple reason label such as due soon, near location, high priority, or suitable time could improve user trust.

The sixth observation is that notification timing still needs refinement. Even though notification usefulness scored positively, some users mentioned delayed reminders, too many reminders in certain situations, or reminders arriving at slightly inconvenient moments. This shows that adaptive notification delivery is useful, but it must be carefully controlled to avoid notification fatigue.

The seventh observation is that long term productivity improvement was not strongly proven. The productivity related category had the lowest mean score of 3.50 out of 5. This does not mean the application failed. It means the evaluation period and sample size were not enough to make a strong claim about long term productivity, cognitive load reduction, or habit formation.

7.3 Discussion

7.3.1 Interpretation of results

The results show that SmartTask successfully demonstrated the proof of concept of a context aware mobile task management application. The completed MVP allowed users to create tasks, match tasks with context, receive adaptive reminders, and complete tasks during a real usage period. The high task completion rate of 87.7 percent shows that users were able to use SmartTask effectively for daily task handling.

The notification results also support the main research problem. Traditional task apps often depend on fixed reminders, which can appear at unsuitable times. SmartTask attempted to improve this by using context signals such as time, location, activity, and device state before displaying reminders. Since most notifications were not ignored or missed, the results suggest that users were more likely to notice reminders when they were connected to their situation.

The positive Likert results also show that the application was accepted by users. The overall mean score of 3.73 out of 5 indicates a generally positive experience. The absence of negative Likert ratings is also important because it shows that users did not

reject the system. Instead, the feedback mainly suggested refinement areas such as better notification control, clearer task ranking, and faster task creation.

The usability results are especially important because SmartTask includes more intelligent behavior than a normal static to do list. If the app had been difficult to use, the context aware features would not have been useful in practice. The usability mean score of 3.88 shows that the interface supported the MVP well enough for daily use.

The context awareness and prioritization result of 3.71 shows that the core concept worked, but it also highlights the need for explanation. Users may not fully trust an automatically ranked task list if the reason for the ranking is hidden. Therefore, future versions of SmartTask can improve user confidence by showing the reason behind task order changes.

The lower productivity and adoption mean score of 3.50 must be interpreted carefully. Productivity improvement usually requires a longer testing period, more participants, and comparison with another task management method. In this evaluation, the result is enough to say that users found SmartTask useful, but it is not enough to claim that SmartTask fully improves long term productivity.

7.3.2 Strengths and limitations

One major strength of SmartTask is that it moves beyond a static task list. Instead of only showing tasks in the order entered by the user, the app considers urgency, context relevance, and interruption conditions. This directly supports the research gap identified in the project proposal, where existing task apps were described as limited by fixed reminders and manual prioritization.

Another strength is that the application was tested with real users from two different user groups, students and working professionals. This helped to collect feedback from people with different daily routines. Students used the app for academic tasks, personal reminders, and location-based activities, while working professionals used it for work related and daily responsibility management.

A further strength is the strong completion result. Completing 185 out of 211 created tasks shows that users did not only install the application, but actually used it to manage and finish tasks. This supports the practical value of the MVP.

The notification system is also a strength. The app delivered reminders based on context rather than only fixed time. The positive notification mean score and the relatively low ignored or missed notification rate show that adaptive reminders were useful to participants.

Privacy and local processing are also strengths of the SmartTask design. The completed app followed an offline first and local data approach, which supports user trust because sensitive task and context information does not need to be shared externally. This matches the design direction described in the project materials.

However, the evaluation also had limitations. The first limitation is the small sample size. Only 8 valid responses were collected, so the results cannot be generalized to all users. Because of this, descriptive analysis was more suitable than advanced statistical testing.

The second limitation is the short testing period. Participants used the app for around two weeks. This was enough to evaluate early usability and MVP usefulness, but not enough to prove long term productivity improvement or long-term behavior change.

The third limitation is the absence of a control group. The evaluation did not directly compare SmartTask with another existing task management application under the same conditions. Therefore, the results show positive SmartTask usage, but they do not prove that SmartTask performs better than all traditional to do list apps.

The fourth limitation is that some results were based on self-reported feedback. User opinions are useful for understanding satisfaction and experience, but they may be affected by personal expectations, memory, or testing conditions.

The fifth limitation is related to context accuracy. Context aware features depend on device sensors, permissions, location accuracy, and background processing. If permissions are denied or location detection is delayed, notification timing can be

affected. This explains some of the feedback about delayed reminders or imperfect context matching.

The final limitation is that the full Data Logging and Insights module was not completed as a final implemented feature. The MVP included task interaction handling and basic state updates, but a complete analytics dashboard with detailed productivity insights was outside the completed implementation. This can be included as an extension by future researchers.

Overall, the results show that SmartTask successfully achieved its MVP objective. The system proved that context aware task prioritization and adaptive reminders can support daily task management. Users found the application easy to use, valued location based and timely reminders, and completed a high percentage of created tasks. At the same time, the findings show that further improvement is needed in notification timing, prioritization explanation, task creation speed, and long term evaluation. Therefore, SmartTask can be presented as a successful context aware task management MVP, while clearly stating that larger scale testing and advanced analytics are required for stronger claims about long term productivity and cognitive load reduction.

7.4 Summary of Each Student's Contribution

This project was completed as an individual project due to unavoidable circumstances. Therefore, the student, De Silva S.J.W., carried out all stages of the project independently, including requirement analysis, system design, application development, testing, user evaluation, data analysis, and final report preparation.

8 BUDGET AND BUDGET JUSTIFICATION

The estimated project budget is shown in Table 5.

Table 5 : Budget details of the project

Category	Item / Description	Estimated Cost (USD)	Justification
Labour	Development (UI/UX, backend, ML, testing) – by student	**0**	Student is the developer; no external labour cost.
Material	Android testing devices (student already owns devices)	**0**	Existing devices cover testing; no extra purchase required.
APIs & Services	Google Maps Platform & related APIs usage	**50**	Low-volume usage estimated (~$50) for map/geofence and sensor API calls.
Software & Tools	Figma free tier, Android Studio free, GitHub free	**0**	All required tools are either free or covered under student license.
Deployment & Hosting	Google Play Developer account fee	**25**	One-time fee to publish the app on Google Play.
Transportation & Miscellaneous	Travel / transit for field testing + incidental costs (data usage, test site coffee, etc.)	**30**	Estimate: ~3 field testing excursions × ≈$10 each for transit/data.
Total Estimated Cost		**$105**	

9 ABILITY OF COMMERCIALIZATION

- **Differentiation:** SmartTask stands out from conventional task managers because it adapts to real-life context, taking into account location, user activity, and calendar events, while intelligently prioritizing tasks. This provides highly personalized productivity support that fixed-time reminder apps cannot offer.
- **Investment & Cost:** Development costs are minimal as the app is created by a student with no labor cost, using existing Android devices with no hardware cost. The main expense is API subscriptions, such as Google Maps costing approximately USD 50 per year for geofencing. Revenue is expected through a freemium model with free ad-supported access and premium subscriptions offering advanced insights and an ad-free experience.
- **Target Market & Customer Profile:** SmartTask targets students, busy professionals, and productivity enthusiasts. These users are mobile-first, heavily rely on task lists, and operate in fast-paced environments. The app helps them improve efficiency without adding cognitive load.
- **Value Proposition:** SmartTask reduces cognitive strain by delivering timely, context-aware notifications rather than generic reminders. This helps users complete tasks on time, manage priorities dynamically, and increase overall productivity in their daily routines.
- **Intellectual Property Considerations:** The proprietary context-matching and adaptive notification algorithms create defensible intellectual property. This prevents competitors from easily replicating SmartTask's core functionality and strengthens its long-term commercial potential.

10 CONCLUSION

10.1 Summary of the Project

The SmartTask project focused on designing and implementing a context aware mobile to do list application for intelligent task management. The main problem addressed by this project was that traditional task management applications mostly depend on static task lists, fixed reminders, and manual prioritization. These limitations can increase cognitive load, cause poorly timed reminders, and lead to notification fatigue.

To address this problem, the SmartTask app was developed as a native Android application. The system uses contextual information such as location, time, user activity, and device state to identify task relevance. Based on this context, the app prioritizes tasks dynamically and delivers reminders at more suitable moments. The system was developed as a minimum viable product, with the main focus on proving the core concept of context aware task filtering, task prioritization, and adaptive notification delivery.

The application includes key functions such as task creation, task editing, task deletion, location-based task handling, context acquisition, task matching, dynamic task ordering, and notification actions such as Done, Snooze, and Postpone. The system also uses local storage to support offline functionality and protect user privacy.

The final evaluation was conducted using system testing and user evaluation. The user evaluation included 8 valid participants who used the application for 12 to 15 days. The results showed that users were able to use SmartTask in daily routines and respond positively to its context aware features.

10.2 Achievements

The main achievement of the project was the successful development of a working context aware task management MVP. The MVP can be downloaded and installed on an Android mobile device using the link provided in Appendix G. The system demonstrated that task relevance can be improved by using real time context instead of depending only on manually entered reminders.

The project achieved the following outcomes:

- A native Android mobile application was developed using Java and Android Studio.
- Users were able to create, edit, delete, complete, snooze, and postpone tasks.
- The app collected contextual information such as location, time, activity, device state, and connectivity.
- A context matching process was implemented to compare user context with task details.
- Tasks were dynamically prioritized based on urgency, context relevance, interruption level, and manual priority.
- Adaptive notifications were delivered only when tasks were relevant and suitable for the user's current situation.
- Notification actions updated task states and influenced later task handling.
- Local database storage was used to support offline use and improve privacy.
- The application was tested using emulator based testing, real device testing, and user evaluation.
- The user evaluation produced positive results, including an overall task completion rate of 87.7 percent and an ignored or missed notification rate of 16.5 percent.

The Likert scale results also showed positive user acceptance. The overall mean score was 3.73 out of 5. Usability and navigation received the highest category mean score of 3.88, while notification usefulness and timing received a mean score of 3.84. These results show that users found the app useful, understandable, and relevant for daily task management.

10.3 Limitations

Although SmartTask successfully achieved its MVP objective, several limitations were identified.

The first limitation is the limited sample size used for user evaluation. Only 8 valid participants were included, so the results cannot be generalized to all users. The results

are useful for evaluating the MVP, but a larger study is needed to produce stronger conclusions.

The second limitation is the short testing duration. Participants used the app for around two weeks. This period was suitable for early evaluation, but it was not enough to fully measure long term productivity improvement, habit formation, or long term cognitive load reduction.

The third limitation is the absence of a control group. SmartTask was not tested directly against another traditional task management application under the same conditions. Therefore, the results show that SmartTask was useful, but they do not prove that it performs better than all existing to do list applications.

The fourth limitation is related to context accuracy. The app depends on device sensors, permissions, location services, and background processing. If permissions are denied, location detection is delayed, or sensor data is inaccurate, context matching and notification timing can be affected.

The fifth limitation is that advanced behavioral learning was not fully implemented. The system used task state changes such as Done, Snooze, and Postpone to affect later task handling, but it did not include a complete learning model that automatically adapts to long term user patterns.

The sixth limitation is that the Data Logging and Insights module was not implemented as a full user facing analytics feature. The initial design included this component, but the final MVP focused on proving the core task management and context aware notification concept.

The final limitation is battery and resource usage. Since context aware systems require background checks and sensor access, some battery usage is expected. Although the app was designed to reduce unnecessary processing, further optimization is needed for long term daily use.

10.4 Future Work

Several improvements can be added to SmartTask in the next stage of development.

A full behavioral learning model can be introduced to learn from user actions over time. This can help the system understand when users usually complete tasks, which reminders they ignore, and which contexts are most useful for different task types.

The Data Logging and Insights module can also be implemented as a future extension. This can provide users with useful summaries such as task completion trends, ignored reminder patterns, most productive time periods, and context based productivity insights.

The prioritization system can be improved by adding explanation labels. For example, the app can show short reasons such as due soon, near location, high priority, or suitable time. This can help users understand why a task appears at the top of the list.

Notification control can be improved by allowing users to set quiet hours, preferred reminder frequency, and notification sensitivity. This can reduce notification fatigue and give users more control over their experience.

The application can also be tested with a larger and more diverse user group. A longer evaluation period with students, working professionals, and other daily task users can provide stronger evidence about long term usefulness and productivity improvement.

Other possible future improvements include cloud backup, cross device synchronization, calendar integration improvements, wearable device support, and a more advanced voice task creation feature.

10.5 Final Evaluation of the Project

Overall, the SmartTask project successfully demonstrated the main concept of a context aware mobile task management system. The completed MVP showed that user context can be used to prioritize tasks and deliver reminders at more suitable moments. The high task completion rate, positive Likert scale results, and user feedback show that the system was useful and practical for daily task management.

The project did not fully prove long term productivity improvement due to the small sample size, short testing period, and lack of a control group. However, it successfully proved that the core idea is feasible and valuable. The SmartTask app provides a strong foundation for future development in context aware productivity systems.

Therefore, the final evaluation of the project is positive. SmartTask achieved its main MVP goal by combining task management, context awareness, dynamic prioritization, and adaptive notifications into a working mobile application. With further improvements such as advanced behavioral learning, full analytics, better notification control, and larger scale testing, SmartTask can be developed into a more complete and reliable intelligent task management solution.

REFERENCES

[1] G. Mark, D. Gudith, and U. Klocke, “The cost of interrupted work: More speed and stress,” *in Proc. ACM CHI*, 2008, pp. 107–110.

[2] V. Gonzalez, V. M. Gonzalez, and G. Mark, “Constant, constant, multitasking craziness: Managing multiple working spheres,” *in Proc. ACM CHI*, 2004, pp. 113–120.

[3] H. Winghart and A. Al Zaidy, “MageMemo: A gamified productivity tool for user engagement,” *J. Inf. Technol., Cybersecurity, and AI*, vol. 1, no. 1, pp. 9–17, 2024.

[4] A. K. Dey, “Understanding and using context,” *Personal and Ubiquitous Computing*, vol. 5, no. 1, pp. 4–7, 2001.

[5] M. Baldauf, S. Dustdar, and F. Rosenberg, “A survey on context-aware systems,” *Int. J. Ad Hoc and Ubiquitous Computing*, vol. 2, no. 4, pp. 263–277, 2007.

[6] A. González-Pérez, I. Herrera-Carnicer, and R. Contreras, “AwarNS: A framework for developing context-aware reactive mobile applications for health and mental health,” *J. Biomed. Inform.*, vol. 141, 2023, Art. 104359.

[7] T. Sohn, A. Varshavsky, D. Chen, R. K. Gupta, and W. G. Griswold, “Place-its: A study of location-based reminders on mobile phones,” *in Proc. ACM UbiComp*, 2005, LNCS 3660, pp. 232–250.

[8] A. Kessell and C. Chan, “Castaway: A context-aware task management system,” *in CHI 2006 Ext. Abstracts*, 2006, pp. 941–946.

[9] B. M. Landry *et al.*, “Taskminder: A context- and user-aware to-do list management system,” Tech. Rep., Georgia Institute of Technology, 2003.

[10] P. Muñoz, P. Giner, and M. Gil, “Designing context-aware interactions for task-based applications,” *in Proc. Int. Conf. on Web Engineering (ICWE)*, 2010, LNCS 6385, pp. 463–473.

[11] B. Chihani, E. Bertin, and N. Crespi, “A user-centric context-aware mobile assistant,” *in Proc. IEEE Int. Conf. on Intelligence in Next Generation Networks (ICIN)*, 2013, pp. 17–21.

[12] C. Zhang *et al.*, “Grounded task prioritization with context-aware sequential ranking,” *ACM Trans. Inf. Syst.*, vol. 40, no. 4, Art. 28, 2022.

[13] Todoist, “A to-do list to organize your work & life,” 2025. [Online]. Available: https://www.todoist.com

[14] Todoist, “Use location reminders in Todoist,” updated Sep. 23, 2025. [Online]. Available: https://www.todoist.com/help/articles/use-location-reminders-in-todoist-uGcwH2AJ6

[15] Any.do, “How to manage notifications in Any.do,” 2023. [Online]. Available: https://support.any.do/en/articles/8610107-how-to-manage-notifications-in-any-do

[16] Microsoft, “Microsoft To Do: Task Management App,” 2025. [Online]. Available: https://www.microsoft.com/en-us/microsoft-365/microsoft-to-do

[17] Google, “Manage Google Tasks notification settings,” 2025. [Online]. Available: https://support.google.com/tasks/answer/13486720

[18] A. Li, “How Google Keep’s switch to Tasks will work: No location reminders,” *9to5Google*, Sep. 6, 2025. [Online]. Available: https://9to5google.com/2025/09/06/google-keep-tasks-switch-locaton

[19] TickTick, “An introduction to TickTick,” Aug. 2, 2016. [Online]. Available: https://blog.ticktick.com/2016/08/02/an-introduction-to-ticktick

[20] Trello, “Capture, organize, and tackle your to-dos from anywhere,” 2025. [Online]. Available: https://trello.com

[21] F. Luqman and M. L. Griss, “Overseer: A mobile context-aware collaboration and task management system for disaster response,” *in Proc. IEEE C5*, 2010.

[22] M. Streefkerk, G.-J. van der Linden, and G. de Haan, “Do(not)Disturb: Effects of context-aware notification management on task performance,” *Int. J. Hum.-Comput. Stud.*, vol. 70, no. 3, pp. 239–245, 2012.

[23] S. T. Iqbal, *A Framework for Intelligent Notification Management in Multitasking Domains*, Ph.D. dissertation, Univ. Illinois at Urbana-Champaign, 2008.

[24] Q. U. Ahmed and S. B. Mujib, “Context aware reminder system: Activity recognition using smartphone accelerometer and gyroscope,” M.Sc. thesis, Blekinge Inst. Technol., Sweden, 2014.

[25] O. A. Faruque, M. Koehl, F. L. Goh *et al.*, “AWARE: A mobile context instrumentation framework,” *Front. ICT*, vol. 2, Art. 6, 2015.

[26] Y. Tian, Z. Wang, D. Pelleg *et al.*, “Characterization and prediction of mobile tasks,” *ACM Trans. Inf. Syst.*, vol. 41, no. 1, 2022.

[27] I. Kim, H. Goh, N. Narziev, Y. Noh, and U. Lee, “Understanding user contexts and coping strategies for context-aware phone distraction management system design,” *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 4, no. 4, Art. 134, 2020.

[28] B. P. Bailey and J. A. Konstan, “Are informational notifications disruptive?,” *in Proc. ACM CHI*, 2006, pp. 331–340.

[29] Bailey, B. P., & Iqbal, S. T. (2008). Understanding changes in mental workload during execution of goal-directed tasks and its application for interruption management. ACM Transactions on Computer-Human Interaction, 14(4), Article 21.

APPENDIX

Appendix - A: Database Design

Table Name : tasks	
property name	**data type**
id	long
title	String
description	String (nullable)
category	String
createdAt	long
updatedAt	long
dueAt	Long (nullable)
priority	int
displayOrder	long
locationLat	Double (nullable)
locationLng	Double (nullable)
locationRadiusM	Float (nullable)
locationLabel	String (nullable)
estimatedDurationMin	Integer (nullable)
preferredStartTime	Long (nullable)
preferredEndTime	Long (nullable)
notificationsEnabled	boolean
completed	boolean
completedAt	Long (nullable)
archived	boolean
snoozeUntil	Long (nullable)

Table Name : context_snapshots	
property name	**data type**
id	long
timestamp	long
timezoneId	String
dayOfWeek	int
minuteOfDay	int
lat	Double (nullable)
lng	Double (nullable)
accuracyM	Float (nullable)
speedMps	Float (nullable)
bearingDeg	Float (nullable)
isGeofenceHit	boolean
geofenceId	String (nullable)
placeLabel	String (nullable)
locationSource	String

activityType	String (nullable)
activityConfidence	int
isMoving	boolean
stepsSinceLast	Integer (nullable)
isInMeeting	boolean
currentEventId	String (nullable)
currentEventTitle	String (nullable)
eventBusyStatus	String (nullable)
minutesToNextEvent	Integer (nullable)
minutesLeftInEvent	Integer (nullable)
batteryPct	int
isCharging	boolean
powerSaveMode	boolean
ringerMode	String (nullable)
doNotDisturbOn	boolean
screenOn	boolean
deviceUnlocked	boolean
appInForeground	boolean
connectivityType	String (nullable)
isInternetAvailable	boolean
isRoaming	Boolean (nullable)
headphonesConnected	Boolean (nullable)
bluetoothConnected	Boolean (nullable)
wifiSsidHash	String (nullable)
noiseLevelDb	Float (nullable)
interruptionCostScore	float
receptivityScore	float
contextLabel	String (nullable)
permissionState	String
dataQualityFlags	int
anonymized	boolean
sourceTrigger	String

Table Name : task_context_matches	
property name	**data type**
id	long
taskId	long
snapshotId	long
matchedAt	long
relevanceScore	float
matchReasons	String
shouldTriggerNow	boolean
cooldownUntil	Long (nullable)
triggerType	String

contextSnapshotTrigger	String (nullable)
blockedBy	String (nullable)

Table Name : usage_stat_events	
property name	**data type**
id	long
eventType	String
taskId	Long (nullable)
notificationId	Integer (nullable)
createdAt	long

Table Name : task_description_insights	
property name	**data type**
id	long
taskId	long
descriptionHash	String
riskIfDelayed	float
requiresDeepFocus	boolean
suggestedDurationMin	Integer (nullable)
complexityLevel	int
energyDemand	int
relationshipValue	int
rawJson	String (nullable)
model	String (nullable)
updatedAt	long

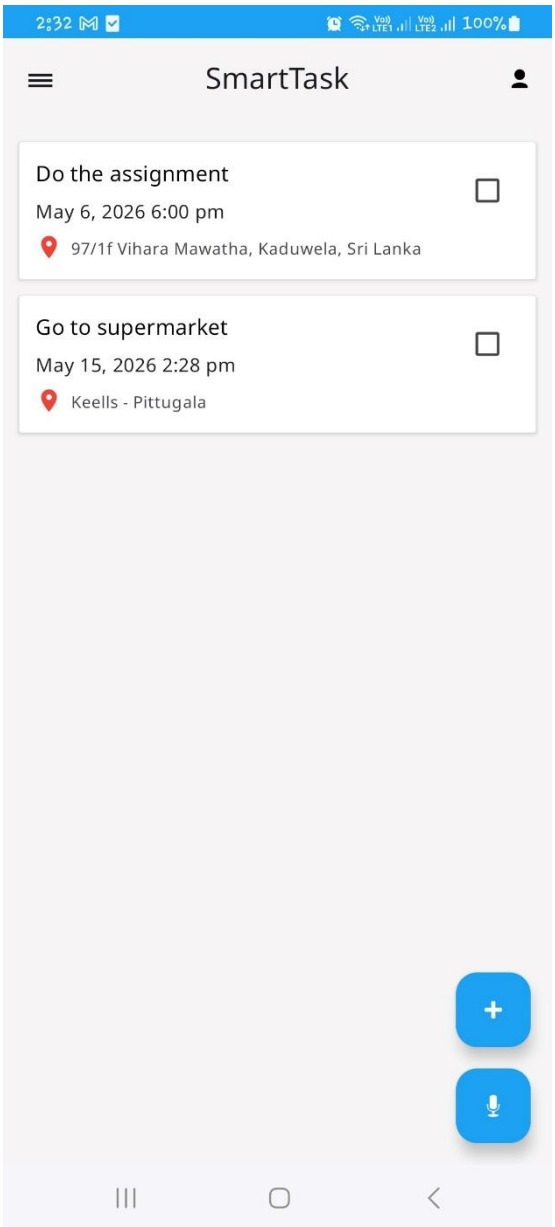
Table Name : voice_command_logs	
property name	**data type**
id	long
transcript	String
extractedJson	String
createdAt	long

Appendix - B: Codebase GitHub Repository Link

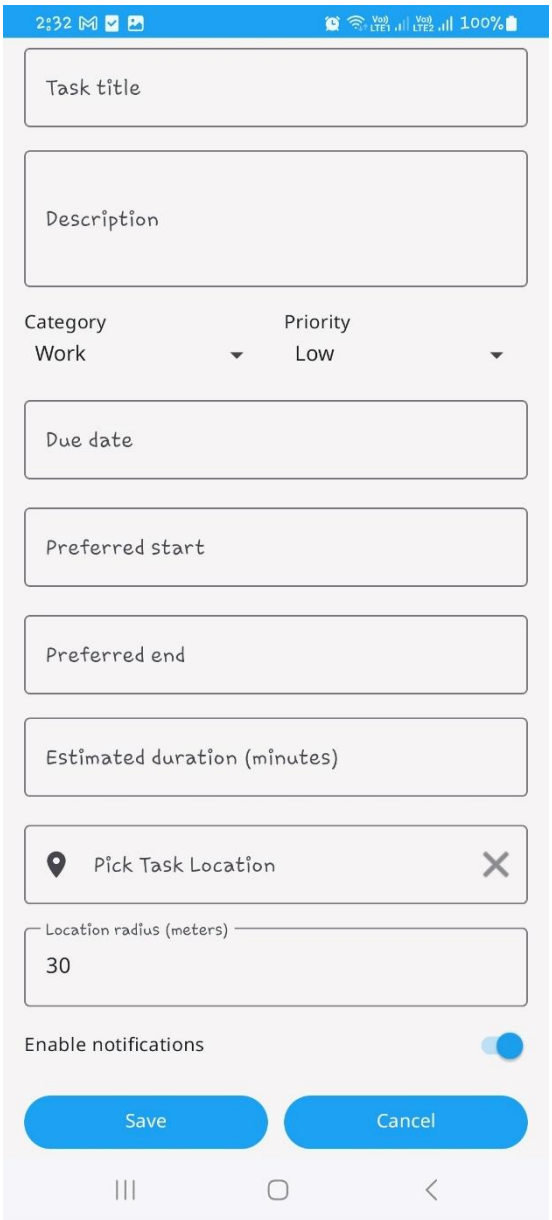
This appendix provides the GitHub repository link for the SmartTask application source code. The repository contains the Android project files, and related implementation files.

Repository Link: https://github.com/IT21228612/SmartTask

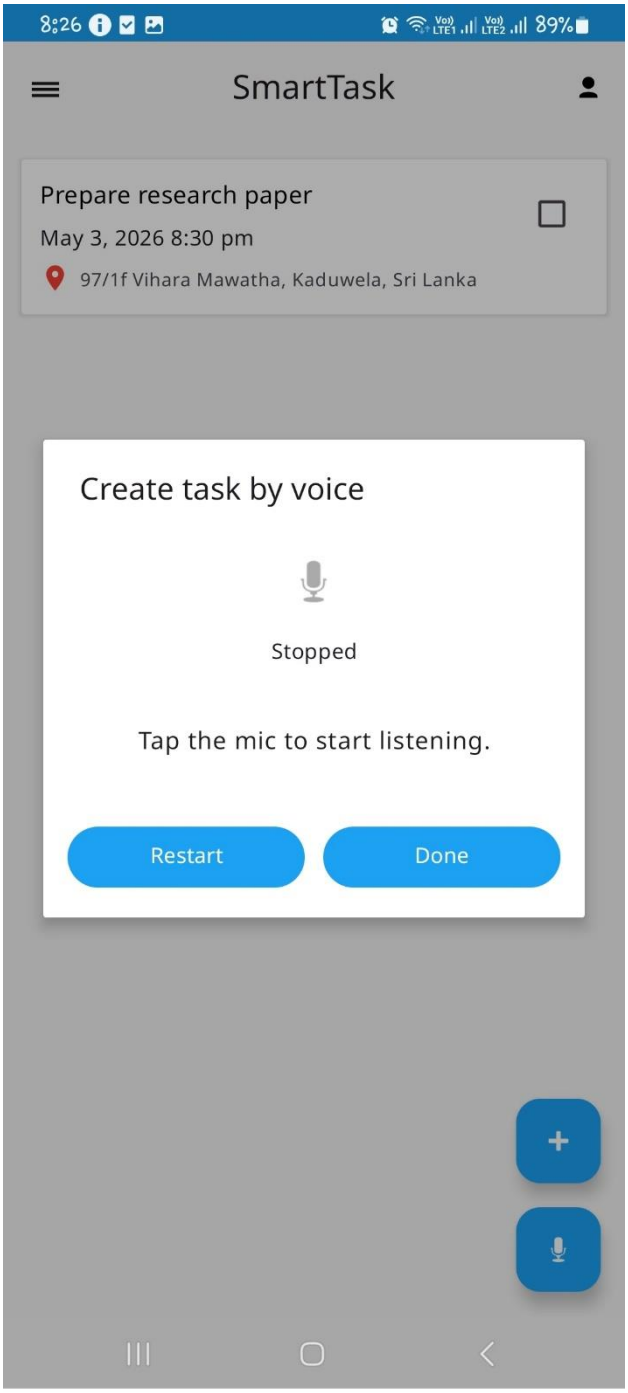
Appendix - C: App UI Screenshots



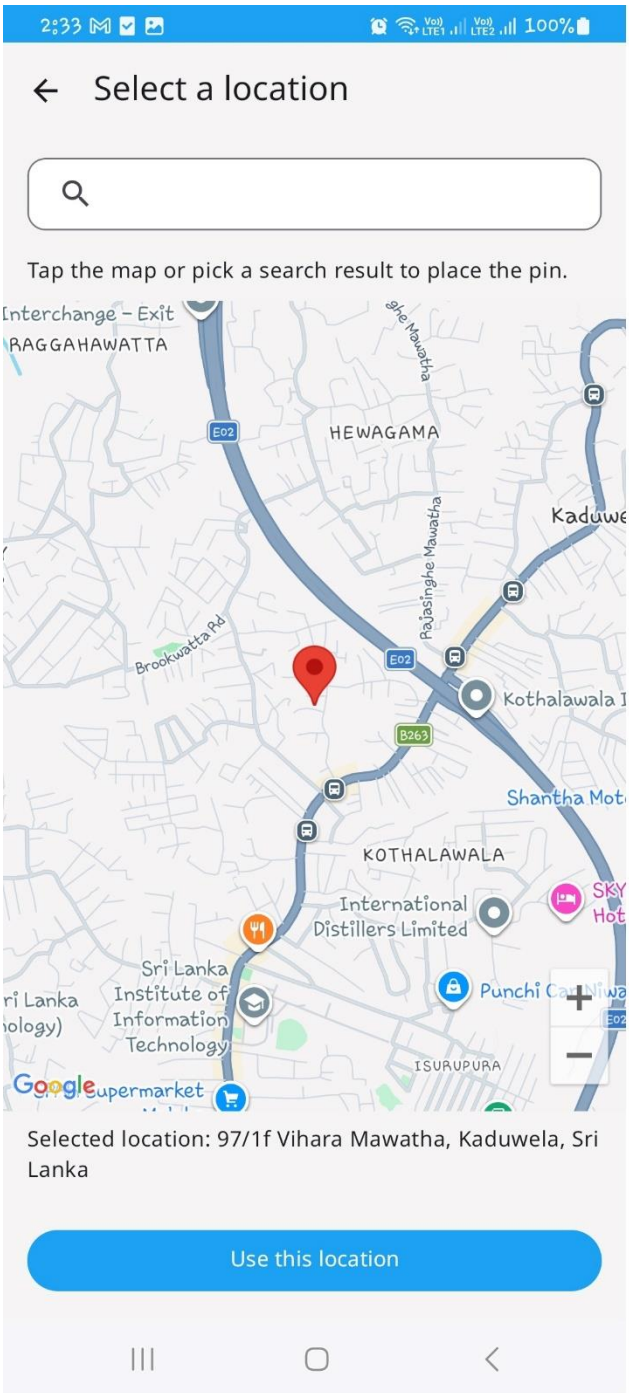
main UI - task list



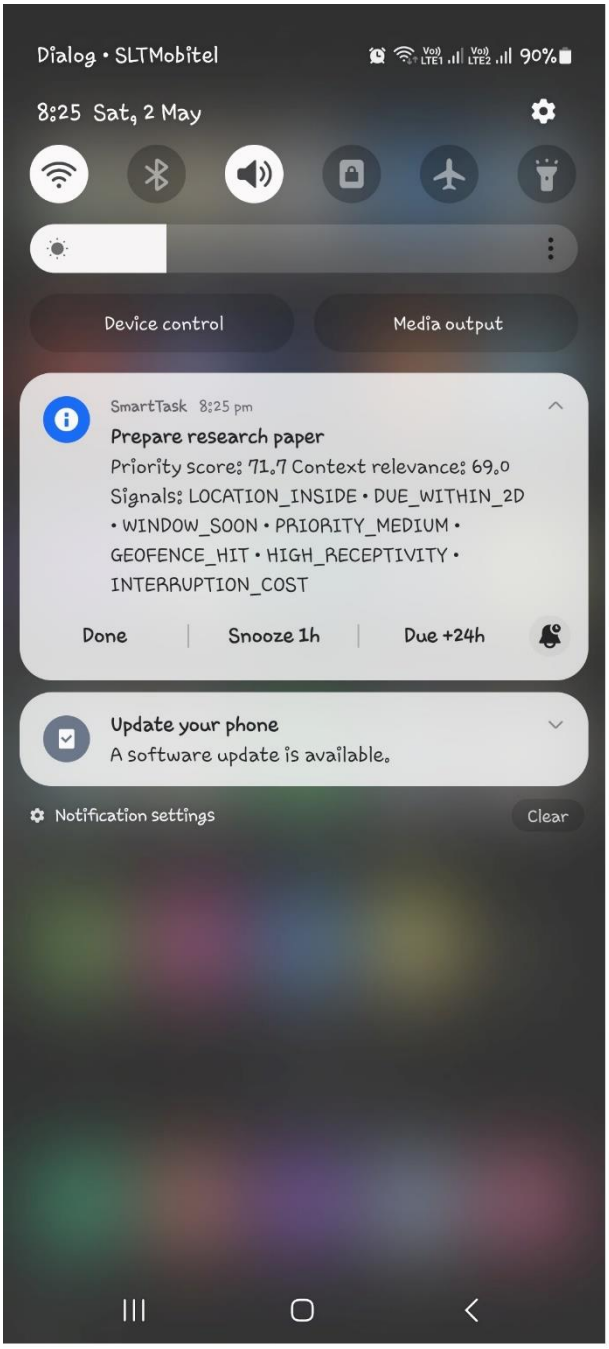
task creation UI



Voice Command Task Creation



Location pick UI



App Notification

Appendix - D: User Evaluation Participation Consent Form

User Participation Consent Form

SmartTask Application Evaluation Study

This document confirms voluntary participation in the evaluation of the **SmartTask: Context-Aware Mobile Task Management Application.**

The purpose of this study is to assess the usability, effectiveness, and user satisfaction of the SmartTask application in real-world usage conditions.

Survey Access

Participants completed the evaluation questionnaire via the link/QR code below:

Scan to access the questionnaire:

Scan me!

Participant Declaration

I confirm that:

- I have voluntarily participated in this study
- I have used the SmartTask application during the testing period
- I have interacted with the application in my daily routine (e.g., creating tasks, responding to notifications)
- I have provided my honest feedback through the online questionnaire
- I understand that my responses will be used only for academic purposes
- I understand that my personal identity will not be disclosed in any report or publication

Scanned with CamScanner

Participant Record

Participant ID	Occupation / Category	Date	Signature
P1	☐ Student ☑ Working Professional ☐ Stay-at-home ☐ Other	29/04/2026	
P2	☐ Student ☑ Working Professional ☐ Stay-at-home ☐ Other	29/04/2026	
P3	☐ Student ☑ Working Professional ☐ Stay-at-home ☐ Other	29/04/2026	
P4	☑ Student ☐ Working Professional ☐ Stay-at-home ☐ Other	30/04/2026	
P5	☑ Student ☐ Working Professional ☐ Stay-at-home ☐ Other	30/04/2026	
P6	☑ Student ☐ Working Professional ☐ Stay-at-home ☐ Other	2026/04/30	
P7	☑ Student ☐ Working Professional ☐ Stay-at-home ☐ Other	2026/04/30	
P8	☐ Student ☑ Working Professional ☐ Stay-at-home ☐ Other	2026-04-30	

Researcher Declaration

The data collected from participants is used strictly for academic evaluation of the SmartTask system. All responses are handled with confidentiality, and no personally identifiable information is included in the research project documents.

Researcher:
De Silva S.J.W.
BSc (Hons) Information Technology (Software Engineering)
Sri Lanka Institute of Information Technology

2026/04/29
Signature and Date

Scanned with CamScanner

Appendix - E: User Evaluation Questionnaire (QR Code)

Appendix - F: User Evaluation Summary – Excel Workbook (Link)

This appendix provides the full Excel workbook used to summarize the SmartTask user evaluation results. The workbook includes the raw questionnaire responses, overall dashboard, usage metrics, Likert scale analysis, open ended feedback summary, and final interpretation notes.

The evaluation was based on 8 valid participant responses collected during the SmartTask testing period. The workbook summarizes key results such as 109 total active testing days, 211 tasks created, 185 tasks completed, an overall task completion rate of 87.7 percent, 194 notifications received, and an ignored or missed notification rate of 16.5 percent.

It also includes the Likert scale category results for usability and navigation, context awareness and prioritization, notification usefulness and timing, and productivity, effort, satisfaction and adoption. In addition, the workbook summarizes open ended feedback themes, including liked features, difficulties faced by users, and suggested improvements for future development.

Excel Workbook Link: https://mysliit-my.sharepoint.com/:x:/g/personal/it21228612_my_sliit_lk/IQBDQKBH8iiGQIdJewT00a_XAUmAtutbaxsw06lOyb5W87k?e=6GdLrv

Appendix - G: APK File and Installation Instructions (Link)

This appendix provides the Cloud Storage folder link that contains the SmartTask MVP APK file and the installation instructions PDF. The APK file allows the developed SmartTask mobile application to be downloaded and installed on an Android device for testing or demonstration purposes. The installation instructions PDF explains the required steps to install the APK safely, including downloading the file, allowing installation from unknown sources if needed, and opening the application after installation.

Folder Link: https://mysliit-my.sharepoint.com/:f:/g/personal/it21228612_my_sliit_lk/IgAKKz4b-E6URI-lQY_ZVyseATm4DoqO9ltlm7y7uBpMhJI?e=4nHRgd